

EvoChartCode: Evolution-Guided Chart Code Induction

This report summarizes the executable EvoChartCode research package: explicit Chart Code extraction, question-conditioned selection, code-grounded reasoning, verification, evolution, ablations, and analysis.

Method

The system converts chart images into structured Chart Code, validates the schema, selects evidence for each question type, reasons over the selected code or image+code input, and verifies the answer against explicit evidence.

Local Results

Method Split EM Relaxed Numeric NA-F1 Invalid Mean Latency
ablation_fixed_chart_code validation 0.2500 0.2812 0.4267 0.0000 0.0003
full_evochartcode_metadata validation 0.2500 0.2812 0.4267 0.0000 0.0003
ablation_no_derived_relations validation 0.2500 0.2812 0.4267 0.0000 0.0003
ablation_no_uncertainty validation 0.2500 0.2812 0.4267 0.0000 0.0003
ablation_no_verifier validation 0.2500 0.2812 0.4267 0.0000 0.0003
code_only validation 0.2500 0.2812 0.4267 0.0000 0.0006
full_evochartcode validation 1.0000 1.0000 0.0000 0.0000 9.7641
full_evochartcode validation 0.5312 0.5781 0.4000 0.0000 13.2813
full_evochartcode validation 0.7500 0.8125 0.8571 0.0000 1.5482

ChartCode-300

{`"benchmark": "ChartCode-300", "split": "validation", "num_charts": 32, "mean_quality_score": 0.8681547619047619, "chart_type_accuracy": 0.65625, "legend_exists_accuracy": 0.23076923076923078, "colorbar_exists_accuracy": 1.0, "image_size_valid_rate": 1.0, "plot_area_present_rate": 1.0, "axis_evidence_present_rate": 0.9375, "series_or_mark_present_rate": 0.90625}`}

Bootstrap And Pareto Analysis

{`"num_runs": 9, "exact_match_ci": {"mean": 0.4201388888888889, "low": 0.25, "high": 0.6111111111111112, "samples": 9}, "relaxed_numeric_ci": {"mean": 0.453125, "low": 0.28125, "high": 0.6336805555555556, "samples": 9}, "na_f1_ci": {"mean": 0.42412698412698413, "low": 0.2814814814814815, "high": 0.5227513227513227, "samples": 9}, "pareto_frontier": [{"num_examples": 1, "exact_match": 1.0, "relaxed_numeric": 1.0, "invalid_rate": 0.0, "na_precision": 0.0, "na_recall": 0.0, "na_f1": 0.0, "mean_latency": 9.764127499991446, "p95_latency": 9.764127499991446, "method": "full_evochartcode", "split": "validation", "policy": {}, "path": "outputs\\runs\\qwen_full_evochartcode_smoke.json"}], {"num_examples": 16, "exact_match": 0.75, "relaxed_numeric": 0.8125, "invalid_rate": 0.0, "na_precision": 1.0, "na_recall": 0.75, "na_f1": 0.8571428571428571, "mean_latency": 1.5482290812524298, "p95_latency": 3.067865599945477, "method": "full_evochartcode", "split": "validation", "policy": {}, "path": "outputs\\runs\\qwen_full_evochartcode_validation16.json"}], {"num_examples": 128, "exact_match": 0.25, "relaxed_numeric": 0.28125, "invalid_rate": 0.0, "na_precision": 0.2711864406779661, "na_recall": 1.0, "na_f1": 0.4266666666666667, "mean_latency": 0.0002748671865901997, "p95_latency": 0.0003100000030826777, "method": "ablation_fixed_chart_code", "split": "validation", "policy": {"verifier": {"min_confidence": 0.4}}, "path": "outputs\\runs\\ablation_fixed_chart_code.json"}]}`}

Limitations

The deterministic metadata/CV backend is a reproducible local path. Paper-quality claims require model-backed Qwen extraction and image+code evaluation on available datasets; missing transfer roots are reported rather than inferred.

Full Paper-Writing Summary

EvoChartCode Summary For Paper Writing

This file is the paper-facing source of truth for the current repository state. It summarizes the idea, method, implementation, experiments, results, artifacts, and remaining claim boundaries for writing a final paper from the local work.

All numeric results below are taken from checked-in output files under `outputs/`, `paper/tables/`, and `paper/figures/` after the completed local runs on May 3, 2026.

0. Proposal-Scale Implementation Update

After the initial local experimental slice, the repository was extended with a checkpointed proposal-scale workflow:

- `scripts/materialize\_full\_transfer.py` materializes full transfer manifests in shards with image checksums.
- `scripts/extract\_transfer\_chart\_codes.py` extracts Qwen Chart Code caches for transfer images.
- `scripts/run\_transfer\_qwen\_full.py` evaluates sharded transfer manifests with cached Qwen Chart Code and image+code reasoning.
- `scripts/run\_qwen\_multiseed.py` runs checkpointed multi-seed Qwen evaluation with bootstrap intervals.
- `scripts/run\_qwen\_ablation\_suite.py` runs eight Qwen-backed ablations.
- `scripts/run\_source\_mutation\_evolution.py` runs local-Qwen source-code mutation in throwaway workspaces.
- `scripts/build\_neurips\_paper.py` writes NeurIPS-style LaTeX assets and compiles `paper/neurips/main.pdf`.
- `reproduce\_proposal\_scale.sh` ties the full checkpointed workflow together.

Completed proposal-scale checks in the current workspace:

Item	Status
ChartQA official test manifest	Ready: 2,500 rows / 2,500 examples / 3 shards
PlotQA official test manifest	Ready: 1,000 rows / 1,000 examples / 2 shards
DVQA full public split	Ready: 200,000 rows / 200,000 images / 2,325,316 QA examples / 2,326 shards
FigureQA full public split	Ready: 100,000 rows / 100,000 images / 1,327,368 QA examples / 1,328 shards
Qwen transfer smoke	Completed for 2 examples per dataset
Qwen ablation smoke	Completed for 8 ablations with limit 8
Qwen multi-seed smoke	Completed for seeds 0 and 1 with limit 2
Local-Qwen source mutation smoke	Ran with 7B and 3B coder models; generated candidates were rejected by validation; a minimal verifier mutation was validated and promoted
NeurIPS LaTeX artifact	Compiled at `paper/neurips/main.pdf`

Important boundary: the full proposal-scale GPU jobs are implemented and checkpointed, but not all are completed in the current output set. The final paper must only claim completed measured outputs.

Proposal-scale smoke metrics:

Smoke run	N	EM	Relaxed	Invalid	Mean latency	Status
ChartQA Qwen transfer	2	0.5	0.5	0.0	4.844278000004124	partial smoke
PlotQA Qwen transfer	2	0.0	0.5	0.0	4.92313144999207	partial smoke
DVQA Qwen transfer	2	1.0	1.0	0.0	4.625680700002704	partial smoke
FigureQA Qwen transfer	2	1.0	1.0	0.0	4.682652200004668	partial smoke
Qwen ablation smoke full EvoChartCode	8	0.625	0.75	0.0	2.3725613624992548	completed smoke
Qwen multi-seed smoke pooled	4	1.0	1.0	0.0	4.632336100003158	completed smoke

Source-mutation smoke details:

- `Qwen/Qwen2.5-Coder-7B-Instruct` loaded but generation failed with a CUDA illegal memory access before a patch was produced.
- `Qwen/Qwen2.5-Coder-3B-Instruct` produced `outputs/source\_mutation\_evolution\_smoke\_3b/patches/g00\_c001.patch`.
- The generated 3B patch was rejected before touching the main workspace because `git apply --check` reported `corrupt patch at line 9`.
- A minimal verifier source mutation was then validated and promoted at `outputs/source\_mutation\_evolution/promoted\_candidate.json`. It broadens exact text containment after normalization and recognizes common horizontal/bottom and vertical/left axis wording. Validation passed `compileall` and an 8-example code-only smoke eval with EM 0.25, relaxed numeric 0.375, invalid rate 0.0, and NA-F1 0.5.

1. Core Idea

EvoChartCode turns chart question answering from direct image-to-answer generation into an explicit, verifiable, evolvable inference pipeline:

```
``text
chart image -> Chart Code -> question-conditioned code selection -> reasoner -> verifier -> answer
````
```

The central thesis is that frozen VLM chart reasoning improves when the model does not answer from pixels alone. Instead, the system first induces an explicit intermediate representation called Chart Code. Chart Code stores visual layout, axes, scales, legends, colorbars, series, marks, derived relations, uncertainty, and provenance. A selector extracts only the evidence relevant to a question. A reasoner answers from that evidence, optionally with the original image. A verifier checks whether the answer is supported by the extracted code and normalizes it for evaluation.

The research framing is:

- Direct VLM chart QA entangles perception, symbolic reasoning, formatting, and abstention.
- Chart Code separates perception from reasoning and gives the system an auditable intermediate object.
- Evolution should optimize more than prompts: schema fields, extractor constraints, serialization, question routing, code selection, reasoning policy, verifier thresholds, and answer normalization can all be searched.
- Verification and uncertainty make the system more reliable, especially for unsupported or ambiguous questions.

## 2. Repository Implementation

The implementation lives in ``evochartcode/``, with experiment drivers in ``scripts/``, configuration in ``configs/``, generated outputs in ``outputs/``, and paper artifacts in ``paper/``.

Important modules:

- ``evochartcode/schema.py``: Pydantic v2 Chart Code schema and coercion utilities.
- ``evochartcode/extractor.py``: metadata/CV extractor and Qwen VLM JSON extractor.
- ``evochartcode/cv.py``: deterministic image metadata and simple CV support.
- ``evochartcode/derived.py``: trend, extrema, turning point, comparison, and related derived relations.
- ``evochartcode/routing.py``: natural-language question routing.
- ``evochartcode/selector.py``: question-conditioned Chart Code subset selection.
- ``evochartcode/serializer.py``: compact serialization for reasoners.
- ``evochartcode/reasoner.py``: code-only and Qwen image+code reasoning.
- ``evochartcode/verifier.py``: support checks, abstention logic, and answer verification.
- ``evochartcode/normalizer.py``: exact-match and numeric answer normalization.
- ``evochartcode/metrics.py``: answer metrics.
- ``evochartcode/evolution.py``: policy mutation, archive, MAP-Elites view, and evolution controller.
- ``evochartcode/quality.py``: Chart Code quality benchmark scoring.
- ``evochartcode/statistics.py``: bootstrap confidence intervals and Pareto frontier analysis.
- ``evochartcode/transfer.py``: transfer dataset manifest discovery.

Important scripts:

- ``scripts/prepare_charxiv.py``: creates chart-level CharXiv splits.
- ``scripts/extract_chart_codes.py``: extracts Chart Code caches.
- ``scripts/run_eval.py``: evaluates code-only or full image+code EvoChartCode.
- ``scripts/run_evolution.py``: runs verification-guided policy evolution.
- ``scripts/run_ablation.py`` and ``scripts/run_ablation_suite.py``: run ablations.
- ``scripts/run_chartcode_quality.py``: evaluates Chart Code quality.
- ``scripts/run_multiseed.py``: multi-seed metadata/code-only evaluation.
- ``scripts/analyze_runs.py``: bootstrap intervals and Pareto analysis over runs.
- ``scripts/materialize_hf_transfer.py``: materializes bounded Hugging Face transfer samples.
- ``scripts/run_transfer_eval.py``: evaluates transfer manifests.
- ``scripts/export_tables.py``: writes paper tables.
- ``scripts/export_figures.py``: writes paper figures.
- ``scripts/build_paper_pdf.py``: builds the generated PDF report.

Primary configs:

- ``configs/charxiv_qwen3vl_2b.yaml``: CharXiv metadata/CV extraction path.
- ``configs/charxiv_qwen3vl_2b_vlm.yaml``: Qwen-backed Chart Code extraction and image+code reasoning path.
- ``configs/evolution_small.yaml``: small local policy evolution run.
- ``configs/evolution_full.yaml``: larger evolution configuration.
- ``configs/ablation_full_evochartcode.yaml``
- ``configs/ablation_fixed_chart_code.yaml``
- ``configs/ablation_no_verifier.yaml``
- ``configs/ablation_no_uncertainty.yaml``
- ``configs/ablation_no_derived_relations.yaml``

## 3. Chart Code Representation

Chart Code is the structured representation induced from each chart. Its purpose is to expose chart evidence in a stable form that can be routed, selected, reasoned over, and verified.

The implemented schema contains:

- ``chart_id``: identifier connecting the chart, image, and questions.
- ``chart_type``: chart family such as line, bar, scatter, heatmap, or unknown.
- ``image_size``: width and height evidence.
- ``plot_area``: estimated plot area geometry.
- ``title``: title text plus confidence.

- ``axes``: x/y or named axis objects containing labels, scale type, range, ticks, and confidence.
- ``legend``: legend existence and item evidence.
- ``colorbar``: colorbar existence, label, ticks, and confidence.
- ``series``: series entries with ids, names, visual encodings, and points.
- ``marks``: optional low-level mark evidence.
- ``text_regions``: optional detected text evidence.
- ``derived_relations``: trends, extrema, turning points, comparisons, and other symbolic relations.
- ``uncertainty``: global and field-level confidence.
- ``provenance``: extraction backend, repair notes, and failure metadata.

The Qwen extractor is intentionally strict: it asks the model for JSON and then repairs common VLM JSON/schema failures before validation. The repair layer handles fenced code blocks, malformed JSON spans, Python-like literals, YAML-like payloads, title strings, missing legend/colorbar objects, axes expressed as lists, primitive tick values, series without ids, point dictionaries, and null uncertainty/provenance fields.

If validation still fails after repair, the extractor writes a low-confidence Chart Code object with a ``validation_repair_error`` provenance entry instead of crashing a long extraction job. This preserves the experiment pipeline while keeping the failure visible.

## 4. Extraction Backends

### 4.1 Metadata/CV Backend

The metadata/CV backend is deterministic and fast. It uses available CharXiv metadata and lightweight image processing to create a valid Chart Code object. It is useful for reproducible local testing, pipeline validation, evolution smoke tests, table export, and CI-like checks.

Its limitation is that it does not truly read chart values from pixels. As a result, metadata/code-only results are useful as a baseline and implementation test, but not as the main paper-quality evidence for chart understanding.

### 4.2 Qwen VLM JSON Backend

The Qwen backend uses cached ``Qwen/Qwen3-VL-2B-Instruct`` weights to extract Chart Code from chart images. It uses CUDA in the local ``vlm`` conda environment. The completed local Qwen extraction run produced Chart Code for 32 CharXiv validation charts covering 128 descriptive validation queries.

Command used:

```
```bash
conda run -n vlm python scripts/extract_chart_codes.py \
--config configs/charxiv_qwen3vl_2b_vlm.yaml \
--split validation \
--output data/cache/chart_codes/charxiv_qwen_vl \
--limit 128
```
```

Completed output:

- Cache directory: ``data/cache/chart_codes/charxiv_qwen_vl``
- Extracted Chart Codes: 32 JSON files
- Covered evaluation budget: 128 CharXiv descriptive validation queries
- Backend: Qwen JSON extraction with schema repair

## 5. Derived Relations

Derived relations are computed from extracted series and point evidence. They are included so the reasoner does not need to rediscover every symbolic relation from raw points.

Implemented derived relation families:

- Monotonic trend direction.
- Local and global extrema.
- Turning points.
- Series comparisons.
- Ranking-oriented evidence.
- Difference and ratio evidence when supported by selected values.

These derived relations are used by the selector and verifier for trend, extrema, comparison, ranking, difference, and ratio questions.

## 6. Question Routing And Selection

The router maps a natural-language question into an internal question type. The selector uses the route to choose a compact subset of Chart Code.

Implemented question families include:

- title
- axis label
- axis range
- tick value
- tick count
- legend count
- legend name
- colorbar
- chart type
- subplot count
- line count
- bar value
- point value
- trend
- extrema
- turning point
- intersection
- comparison
- ranking
- difference
- ratio
- not applicable
- open ended

The selection step prevents the reasoner from seeing a large, noisy full JSON document when the question only requires a few fields. This is important for both accuracy and latency.

## 7. Reasoning Modes

Two reasoning modes are implemented.

### 7.1 Code-only Reasoning

The code-only reasoner answers from selected Chart Code without looking at the image. This path is deterministic, fast, and interpretable. It is the default path for reproducible local workflow and metadata experiments.

### 7.2 Full Image+Code EvoChartCode

The full `full\_evochartcode` path gives Qwen both the original chart image and the selected Chart Code. This lets the VLM use structured evidence while still consulting pixels when the code is incomplete.

Completed command:

```
`` bash
conda run -n vlm python scripts/run_eval.py \
--config configs/charxiv_qwen3vl_2b_vlm.yaml \
--method full_evochartcode \
--split validation \
--limit 128 \
--output outputs/runs/qwen_full_evochartcode_validation128.json \
--save-predictions outputs/runs/qwen_full_evochartcode_validation128_predictions.json
``
```

Completed output:

- Metrics: `outputs/runs/qwen\_full\_evochartcode\_validation128.json`
- Predictions: `outputs/runs/qwen\_full\_evochartcode\_validation128\_predictions.json`
- Examples: 128
- Exact match: 0.53125
- Relaxed numeric accuracy: 0.578125
- Invalid rate: 0.0
- Not Applicable precision: 1.0
- Not Applicable recall: 0.25
- Not Applicable F1: 0.4
- Mean latency: 13.281345264844276 seconds
- p95 latency: 16.10223769989928 seconds

## 8. Verifier And Normalizer

The verifier checks whether an answer is supported by selected Chart Code. The normalizer makes answers comparable under exact-match

and relaxed numeric metrics.

Implemented verifier coverage:

- title support
- axis label support
- axis range support
- tick value and tick count support
- legend count/name support
- colorbar support
- chart type support
- subplot count support
- line count support
- bar and point value support
- trend support
- extrema support
- turning point support
- intersection support
- comparison support
- ranking support
- difference support
- ratio support
- Not Applicable support
- open-ended fallback support

The verifier is still rule-based. Its reliability depends on whether the extractor captured the right evidence. If the Chart Code is incomplete or wrong, the verifier can accept an answer that is locally supported by flawed evidence or reject an answer that is visually correct but absent from the code.

## 9. Evolution Framework

The implemented evolution framework searches over policy dictionaries that affect multiple blocks:

- schema optional fields
- extractor prompt constraints and uncertainty inclusion
- selector serialization budget and selected blocks
- reasoner mode and answer length
- verifier thresholds and rule families

The run maintains:

- A program archive of evaluated candidates.
- A MAP-Elites view over behavior bins.
- Behavior descriptors for accuracy, latency, and abstention.
- Mutation summaries for each candidate.
- A best policy output.

Completed small run:

- Output directory: ``outputs/evolution_small``
- Archive: ``outputs/evolution_small/archive.json``
- MAP-Elites map: ``outputs/evolution_small/map_elites.json``
- Best policy: ``outputs/evolution_small/best_policy.json``

Best policy record:

- Candidate: ``g00_c001``
- Parent: ``seed``
- Exact match: 0.265625
- Relaxed numeric accuracy: 0.34375
- Not Applicable F1: 0.4383561643835616
- Invalid rate: 0.0
- Mean latency: 0.0002781468751891225 seconds
- Score: 0.26561109265624055
- Behavior: low accuracy, fast latency, mid abstention
- Mutation summary: mutated ``reasoner.max_answer_tokens``

Important claim boundary: the current evolution run is a policy-level mutation and MAP-Elites implementation. It does not yet contain a production external-LLM code rewriting loop that edits Python source files and validates patches. For the current paper draft, claims should say "verification-guided policy evolution with MAP-Elites-style diversity tracking," not "fully autonomous LLM code mutation" unless that additional experiment is added later.

## 10. Datasets

### 10.1 CharXiv

CharXiv is the primary local dataset used by this repository. The split file is:

```
```text
data/splits/charxiv_chart_level_split.json
```
```

The split sizes are:

```
Split	Charts
evolution_dev	400
validation	300
heldout	300
```

The completed main experiments use CharXiv descriptive validation queries.

### 10.2 Transfer Datasets

The repository includes scripts for transfer data preparation and evaluation:

```
- `scripts/materialize_hf_transfer.py`
- `scripts/prepare_chartqa.py`
- `scripts/prepare_plotqa.py`
- `scripts/prepare_dvqa.py`
- `scripts/prepare_figureqa.py`
- `scripts/run_transfer_eval.py`
```

Public Hugging Face samples were materialized for:

```
- ChartQA from `HuggingFaceM4/ChartQA`
- PlotQA from `achang/plot_qa`
- DVQA from `sionic-ai/dvqa`
- FigureQA from `sionic-ai/figureqa`
```

The completed transfer experiments use 32 examples per dataset and the metadata/code-only path. PlotQA is represented through the available chart-to-table style data in the public dataset sample, so the sample query is generic chart data extraction rather than the full original PlotQA benchmark setup.

## 11. Metrics

Implemented answer metrics:

- Exact match.
- Relaxed numeric accuracy.
- Invalid output rate.
- Not Applicable precision.
- Not Applicable recall.
- Not Applicable F1.
- Mean latency.
- p95 latency.

Implemented quality and analysis metrics:

- Chart Code quality score.
- Chart type accuracy.
- Legend existence accuracy.
- Colorbar existence accuracy.
- Valid image size rate.
- Plot area present rate.
- Axis evidence present rate.
- Series or mark evidence present rate.
- Bootstrap confidence intervals.
- Accuracy-latency Pareto frontier.

## 12. Main Results

### 12.1 CharXiv Metadata/Code-only Baseline

File: `outputs/runs/code\_only\_validation.json`

```
Metric	Value
Examples	128
Exact match	0.25
Relaxed numeric accuracy	0.28125
Invalid rate	0.0
Not Applicable precision	0.2711864406779661
Not Applicable recall	1.0
Not Applicable F1	0.4266666666666667
Mean latency	0.0006176242184210423 s
p95 latency	0.0008824999968055636 s
```

Interpretation: the metadata/code-only baseline is extremely fast and never produces invalid formatted outputs, but accuracy is limited because it cannot recover visual evidence absent from metadata-derived Chart Code.

12.2 Full Qwen EvoChartCode On CharXiv Validation128

File: `outputs/runs/qwen\_full\_evochartcode\_validation128.json`

```
Metric	Value
Examples	128
Exact match	0.53125
Relaxed numeric accuracy	0.578125
Invalid rate	0.0
Not Applicable precision	1.0
Not Applicable recall	0.25
Not Applicable F1	0.4
Mean latency	13.281345264844276 s
p95 latency	16.102237699989928 s
```

Interpretation: Qwen-backed full EvoChartCode substantially improves answer accuracy over the metadata/code-only path on the same 128-query validation budget. Exact match improves from 0.25 to 0.53125. Relaxed numeric accuracy improves from 0.28125 to 0.578125. The cost is much higher latency because the model performs image+code reasoning on GPU.

12.3 Earlier Qwen Smoke And 16-query Validation Runs

File: `outputs/runs/qwen\_full\_evochartcode\_smoke.json`

```
Metric	Value
Examples	1
Exact match	1.0
Relaxed numeric accuracy	1.0
Invalid rate	0.0
Mean latency	9.764127499991446 s
```

File: `outputs/runs/qwen\_full\_evochartcode\_validation16.json`

```
Metric	Value
Examples	16
Exact match	0.75
Relaxed numeric accuracy	0.8125
Invalid rate	0.0
Not Applicable precision	1.0
Not Applicable recall	0.75
Not Applicable F1	0.8571428571428571
Mean latency	1.5482290812524298 s
p95 latency	3.0678655999945477 s
```

Interpretation: the 16-query run is useful as a smoke/early validation result but should not be treated as the main paper result because the sample is small.

13. Ablation Results

The completed ablation suite uses the metadata backend over 128 validation examples. These ablations validate that the experiment runner, configuration system, metrics, and table export paths work. They are not strong evidence of component value because the metadata backend lacks rich visual evidence, so several ablations tie.

```
Run file	Method	Examples	EM	Relaxed Numeric	NA-F1	Invalid	Mean Latency
`outputs/runs/ablation_full_evochartcode.json`	full_evochartcode_metadata	128	0.25	0.28125	0.4266666666666667	0.0	0.00027627578128885943
```

```
`outputs/runs/ablation_fixed_chart_code.json`	ablation_fixed_chart_code	128	0.25	0.28125	0.4266666666666667	
0.0	0.0002748671865901997					
`outputs/runs/ablation_no_verifier.json`	ablation_no_verifier	128	0.25	0.28125	0.4266666666666667	0.0
0.0002757375009423413						
`outputs/runs/ablation_no_uncertainty.json`	ablation_no_uncertainty	128	0.25	0.28125	0.4266666666666667	
0.0	0.0002767195314845594					
`outputs/runs/ablation_no_derived_relations.json`	ablation_no_derived_relations	128	0.25	0.28125		
0.4266666666666667 | 0.0 | 0.0002838140622998253 |
```

Interpretation: this suite is complete as an executable local ablation suite. For a final NeurIPS-level claim, the ablations should be repeated with Qwen-backed Chart Code and full image+code reasoning.

## 14. Chart Code Quality Results

### 14.1 Metadata ChartCode-300

File: `outputs/quality/chartcode\_300.json`

```
Metric	Value
Charts	300
Mean quality score	0.5770952380952381
Chart type accuracy	0.8233333333333334
Legend existence accuracy	0.2962962962962963
Colorbar existence accuracy	0.8654970760233918
Image size valid rate	1.0
Plot area present rate	1.0
Axis evidence present rate	0.0
Series or mark evidence present rate	0.0
```

Interpretation: metadata ChartCode-300 produces valid structural objects and often knows chart type/colorbar metadata, but it lacks true axis and series evidence.

### 14.2 Qwen Chart Code Quality On 32 Cached Validation Charts

File: `outputs/quality/chartcode\_qwen\_validation32.json`

```
Metric	Value
Charts	32
Mean quality score	0.8681547619047619
Chart type accuracy	0.65625
Legend existence accuracy	0.23076923076923078
Colorbar existence accuracy	1.0
Image size valid rate	1.0
Plot area present rate	1.0
Axis evidence present rate	0.9375
Series or mark evidence present rate	0.90625
```

Interpretation: Qwen Chart Code has much stronger axis and series/mark evidence than the metadata backend. The quality benchmark also shows remaining weaknesses in chart type and legend existence extraction.

## 15. Multi-seed, Bootstrap, And Pareto Analysis

### 15.1 Multi-seed Metadata Run

File: `outputs/analysis/multiseed\_code\_only.json`

The metadata/code-only path was run over seeds 0, 1, and 2. Because the path is deterministic, the exact-match and relaxed-numeric intervals collapse to single values.

```
Metric	Mean	Low	High
Exact match	0.25	0.25	0.25
Relaxed numeric accuracy	0.28125	0.28125	0.28125
Not Applicable F1	0.4266666666666667	0.4266666666666667	0.4266666666666667
```

### 15.2 Aggregate Run Analysis

File: `outputs/analysis/run\_analysis.json`

After the 128-query Qwen run was added, aggregate analysis covered 9 run files.

```
Aggregate	Value
```

## | Number of runs | 9 |

```
Exact-match bootstrap mean	0.4201388888888889
Exact-match CI low	0.25
Exact-match CI high	0.6111111111111112
Relaxed-numeric bootstrap mean	0.453125
Relaxed-numeric CI low	0.28125
Relaxed-numeric CI high	0.6336805555555556
NA-F1 bootstrap mean	0.42412698412698413
NA-F1 CI low	0.2814814814814815
NA-F1 CI high	0.5227513227513227
```

The Pareto frontier includes:

- Qwen smoke run: highest measured EM but only 1 example.
- Qwen validation16 run: strong small-sample accuracy and medium latency.
- Metadata ablation fixed Chart Code run: fastest representative validation128 run.

Interpretation: aggregate analysis is useful for repository reporting and visualization. It is not a replacement for a controlled multi-seed Qwen experiment.

## 16. Transfer Results

All completed transfer runs use 32 examples per dataset and the metadata/code-only path.

```
Dataset	File	Examples	EM	Relaxed Numeric	Invalid	Mean Latency
ChartQA	`outputs/transfer/chartqa_code_only.json`	32	0.0	0.0	0.0	0.016100634374197398
PlotQA	`outputs/transfer/plotqa_code_only.json`	32	0.0	0.0	0.0	0.025692524998703448
DVQA	`outputs/transfer/dvqa_code_only.json`	32	0.0	0.0	0.0	0.01730339062441999
FigureQA	`outputs/transfer/figureqa_code_only.json`	32	0.0	0.0	0.0	0.01812600624998595
```

Interpretation: these runs verify dataset materialization, manifest format, transfer evaluation, and metric computation. They do not show successful cross-dataset reasoning because the metadata/code-only path does not extract the visual content needed for these external QA tasks. A final paper should either present these as pipeline transfer checks or add Qwen-backed transfer extraction and reasoning before making cross-dataset accuracy claims.

## 17. Paper Artifacts

Generated paper artifacts:

- Main results table: `paper/tables/main\_results.md`
- Accuracy-latency figure: `paper/figures/accuracy\_latency.svg`
- Generated report PDF: `paper/evochartcode\_report.pdf`
- Draft paper markdown: `paper/draft.md`

The PDF is generated from local outputs. It is a report artifact, not a final camera-ready NeurIPS submission.

## 18. Reproduction Commands

Main deterministic local workflow:

```
```bash
bash reproduce_main.sh
```
```

This runs extraction, code-only evaluation, small evolution, ablations, quality, analysis, Hugging Face transfer sample preparation/evaluation, table export, figure export, and PDF export.

Qwen smoke path:

```
```bash
bash reproduce_qwen_smoke.sh
```
```

Completed Qwen validation128 extraction:

```
```bash
conda run -n vlm python scripts/extract_chart_codes.py \
--config configs/charxiv_qwen3vl_2b_vlm.yaml \
--split validation \
--output data/cache/chart_codes/charxiv_qwen_vl \
--limit 128
```
```

Completed Qwen validation128 evaluation:

```
```bash
```

```
conda run -n vlm python scripts/run_eval.py \
--config configs/charxiv_qwen3vl_2b_vlm.yaml \
--method full_evochartcode \
--split validation \
--limit 128 \
--output outputs/runs/qwen_full_evochartcode_validation128.json \
--save-predictions outputs/runs/qwen_full_evochartcode_validation128_predictions.json
```
```

Qwen quality over cached validation charts:

```
```bash
conda run -n vlm python scripts/run_chartcode_quality.py \
--config configs/charxiv_qwen3vl_2b_vlm.yaml \
--split validation \
--limit 32 \
--output outputs/quality/chartcode_qwen_validation32.json
```
```

Refresh analysis, tables, figures, and PDF:

```
```bash
python scripts/analyze_runs.py --runs outputs/runs --output outputs/analysis/run_analysis.json
python scripts/export_tables.py --runs outputs/runs --out paper/tables
python scripts/export_figures.py --runs outputs/runs --out paper/figures
python scripts/build_paper_pdf.py \
--out paper/evochartcode_report.pdf \
--quality outputs/quality/chartcode_qwen_validation32.json \
--analysis outputs/analysis/run_analysis.json
```
```

Hugging Face transfer sample workflow:

```
```bash
bash reproduce_hf_transfer.sh
```
```

## 19. Suggested Paper Claims Supported By Current Evidence

The following claims are supported by current local artifacts:

1. EvoChartCode implements an explicit Chart Code intermediate representation for chart reasoning.
2. The system separates extraction, question routing, evidence selection, reasoning, verification, and normalization.
3. Qwen-backed Chart Code extraction and schema repair work on a 32-chart CharXiv validation sample covering 128 descriptive queries.
4. Full image+code EvoChartCode improves over metadata/code-only reasoning on the same 128-query validation budget: EM 0.53125 vs 0.25 and relaxed numeric 0.578125 vs 0.28125.
5. Qwen-backed Chart Code has much stronger axis and series/mark evidence than metadata Chart Code on the measured quality samples.
6. The implementation supports verifier coverage for all currently routed question families.
7. The repository includes executable ablation, evolution, multi-seed, bootstrap, Pareto, transfer, table, figure, and PDF workflows.
8. Public Hugging Face samples for ChartQA, PlotQA, DVQA, and FigureQA can be materialized and evaluated through a common transfer manifest format.

## 20. Claims That Need Additional Runs Before A Final NeurIPS Submission

The following claims should not be stated as final paper results unless more experiments are run:

1. Full benchmark-size cross-dataset transfer accuracy on ChartQA, PlotQA, DVQA, and FigureQA.
2. Multi-seed Qwen-backed GPU confidence intervals.
3. Full validation/test-scale Qwen extraction over every CharXiv chart and all question families.
4. Production LLM-mutated source-code evolution rather than policy-level mutation.
5. Strong component ablations under Qwen-backed full EvoChartCode.
6. A camera-ready NeurIPS PDF with final tables, finalized figures, related work, and statistical significance analysis.

This boundary matters because the current repository contains real, reproducible experiments, but not every full-scale result imagined in the proposal.

## 21. Recommended Final Paper Structure

Recommended title:

```
```text
EvoChartCode: Evolution-Guided Chart Code Induction for Reliable Chart Reasoning
```
```

...

Recommended abstract skeleton:

- 1. Direct VLM chart QA is brittle because perception, reasoning, answer formatting, and abstention are entangled.
- 2. EvoChartCode converts chart images into explicit Chart Code containing layout, axes, legends, series, derived relations, uncertainty, and provenance.
- 3. A question-conditioned selector exposes relevant evidence to a code-grounded reasoner.
- 4. A verifier checks answer support and improves reliability.
- 5. Verification-guided evolution searches over representation and inference-policy choices.
- 6. On local CharXiv validation experiments, Qwen-backed full EvoChartCode improves over a metadata/code-only pipeline while preserving valid answer formatting and interpretability.

Recommended sections:

- 1. Introduction.
- 2. Related Work.
- 3. Chart Code Representation.
- 4. EvoChartCode Pipeline.
- 5. Verification-Guided Evolution.
- 6. Experimental Setup.
- 7. Main CharXiv Results.
- 8. Chart Code Quality.
- 9. Ablations And Analysis.
- 10. Transfer Pipeline.
- 11. Limitations.
- 12. Conclusion.

22. Main Tables To Include

Table 1: Main CharXiv Results

Use `paper/tables/main\_results.md`, but in the final paper separate smoke/small-sample Qwen runs from the main 128-query run.

The key comparison to highlight:

| Method                 | Examples | EM      | Relaxed Numeric | Invalid | Mean Latency          |
|------------------------|----------|---------|-----------------|---------|-----------------------|
| ---                    | ---      | ---     | ---             | ---     | ---                   |
| Metadata/code-only     | 128      | 0.25    | 0.28125         | 0.0     | 0.0006176242184210423 |
| Qwen full EvoChartCode | 128      | 0.53125 | 0.578125        | 0.0     | 13.281345264844276    |

Table 2: Chart Code Quality

| Backend  | Charts | Quality            | Axis Evidence | Series/Mark Evidence |
|----------|--------|--------------------|---------------|----------------------|
| ---      | ---    | ---                | ---           | ---                  |
| Metadata | 300    | 0.5770952380952381 | 0.0           | 0.0                  |
| Qwen     | 32     | 0.8681547619047619 | 0.9375        | 0.90625              |

Table 3: Metadata Ablations

Use the ablation table in section 13. Present it as an implementation validation suite unless Qwen ablations are added.

Table 4: Transfer Pipeline Checks

Use the transfer table in section 16. Present these as transfer materialization/evaluation checks, not as evidence that the metadata path solves transfer QA.

23. Main Figures To Include

Available generated figure:

- `paper/figures/accuracy\_latency.svg`

Recommended additional figures to create for the final paper:

- EvoChartCode pipeline diagram.
- Example chart image with extracted Chart Code fields.
- Schema repair and verifier flow.
- MAP-Elites archive visualization.
- Error attribution chart once more failure labels are collected.

## 24. Error Analysis From Current Results

Observed weaknesses:

- Metadata extraction lacks true axis and series values, causing many questions to become unsupported or guessed.
- Qwen extraction improves axis and series evidence but still has chart type and legend errors.
- Qwen image+code reasoning improves accuracy but increases latency by orders of magnitude.
- Not Applicable precision in the Qwen 128 run is high, but recall is low, meaning abstentions are conservative and miss some unsupported cases.
- External transfer sample accuracy is zero under metadata/code-only evaluation because required visual evidence is absent from the extracted code.

Most likely next improvements:

- Improve Qwen extractor prompt and schema repair around legends and chart type.
- Run Qwen-backed ablations to measure verifier, uncertainty, and derived relation contributions under a meaningful extractor.
- Add stronger value extraction for axes, ticks, and plotted points.
- Run transfer evaluation with Qwen-backed Chart Code, not metadata Chart Code.
- Add controlled error labels for perception, reasoning, verifier, and formatting failures.

## 25. Final Status Matrix

```
Proposal item	Current repository status
Explicit Chart Code schema	Implemented in `evochartcode/schema.py`
Qwen-backed Chart Code extraction	Completed for 32 validation charts / 128 descriptive validation queries
Full image+code EvoChartCode GPU evaluation	Completed for 128 CharXiv descriptive validation queries
ChartQA preparation	Completed for bounded HF sample and full official test manifest
PlotQA preparation	Completed for bounded HF sample and full official test manifest
DVQA preparation	Completed for bounded HF sample and full public train manifest
FigureQA preparation	Completed for bounded HF sample and full public train manifest
Transfer results	Completed for 32-example metadata/code-only samples per dataset
MAP-Elites-style evolution	Implemented and run as policy mutation with archive and behavior bins
External LLM source-code mutation	Not completed; current evolution is policy-level mutation
Verifier coverage	Implemented for all current routed question families
ChartCode-300	Completed with metadata backend on 300 validation charts
Qwen Chart Code quality	Completed on 32 cached Qwen validation charts
Multi-seed experiments	Completed for metadata/code-only path
Bootstrap confidence intervals	Completed in `outputs/analysis/run_analysis.json`
Pareto frontier analysis	Completed in `outputs/analysis/run_analysis.json`
Ablation suite	Completed for metadata backend
Qwen-backed ablation suite	Not completed
NeurIPS-style PDF artifact	Generated report exists at `paper/evochartcode_report.pdf`
Final camera-ready paper	Not completed; this summary and generated report are the writing foundation
```

## 26. Bottom Line

The current repository now contains a complete executable EvoChartCode system and a substantial local experimental record:

- Deterministic metadata workflow for reproducibility.
- Qwen-backed Chart Code extraction and full image+code reasoning on 128 CharXiv validation queries.
- Chart Code quality benchmarking.
- Transfer dataset sample materialization and evaluation.
- Policy evolution with MAP-Elites-style archive.
- Ablation, multi-seed, bootstrap, Pareto, table, figure, and PDF generation.

The strongest current empirical result is the CharXiv validation128 comparison:

```
```text
Metadata/code-only: EM 0.25000, relaxed numeric 0.28125, mean latency 0.00062s
Qwen full EvoChartCode: EM 0.53125, relaxed numeric 0.57813, mean latency 13.28135s
```
```

This supports the paper's main direction: explicit, verifiable Chart Code plus image+code reasoning can improve frozen VLM chart QA over a code-only metadata baseline, while making the reasoning path inspectable. For a final NeurIPS-strength submission, the next required work is not basic implementation but scale and rigor: Qwen-backed full ablations, full-size transfer benchmarks, multi-seed Qwen runs, stronger source-code evolution, and polished paper writing.